

Lista de exercícios — Grafos (SALA)

Estrutura base em C:

```
typedef struct {
    int arestanula;
    int maxvertices;
    int n; // número atual de vértices
    int *v; // vetor de vértices
    int **M; // matriz de adjacências
} Grafo;
```

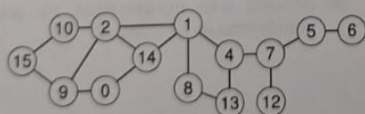
1. Um segurança precisa verificar, no final do dia, os espaços de um museu: **Entrada (E)**, **Recepção (R)**, **Sala A (A)**, **Sala B (B)**, **Sala C (C)** e **Sala D (D)**.

Os corredores entre os espaços são:

$E - R, R - A, A - B, B - R,$
 $R - C, C - D, D - R.$

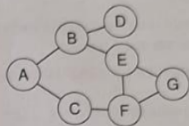
O segurança deve iniciar e terminar o percurso na **Entrada (E)**, passando por todos os corredores exatamente uma vez. Modele a situação como um grafo e justifique se é possível realizar esse percurso.

2. Considere o grafo não direcionado abaixo.



Determine se existe caminho possível entre os vértices **15** e **7** através da Busca em Largura. Sempre que houver mais de uma possibilidade de escolha, considere primeiro o vértice de **menor valor numérico**.

3. Considere o grafo não direcionado abaixo:

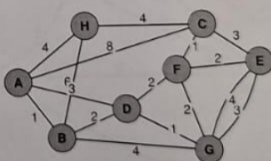


Realize uma busca em profundidade a partir do vértice **A**. Sempre que houver mais de um vizinho disponível, escolha sempre em ordem alfabética.

Determine:

- a ordem em que os vértices são visitados;
- a árvore de busca gerada;
- o caminho encontrado de **A** até **G**.

4. Considere o grafo ponderado não direcionado abaixo.



Determine o menor caminho entre os vértices **H** e **G**.

Apresente:

- a distância mínima entre **H** e **G**;
- o caminho encontrado;
- a atualização das distâncias ao longo da execução do algoritmo.

5. Considere um grafo com n vértices, representado por uma matriz de adjacências M .

Implemente uma função que, usando busca em profundidade no formato recursivo, conte quantos vértices podem ser alcançados a partir de um vértice inicial.

Considere que o vértice de origem é um índice válido, pertencente ao grafo e também deve ser contado.

```
int contar_alcancaveis(Grafo *g,
    int origem);
```

6. Considere um grafo não direcionado com n vértices, representado por uma matriz de adjacências M .

Implemente uma função que verifique se um determinado vértice é isolado.

Utilize a estrutura e a assinatura abaixo:

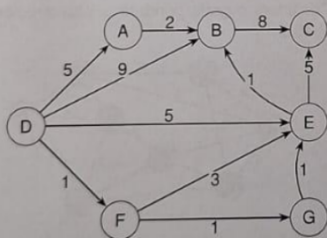
```
int eh_isolado(Grafo *g, int vertice);
```

A função deve retornar:

- 1, se o vértice for isolado;
- 0, caso contrário.

7. Considere o dígrafo ponderado abaixo e a aplicação do algoritmo de Dijkstra para calcular os menores caminhos a partir do vértice **D**.

Considere as seguintes arestas direcionadas com seus respectivos pesos:



Determine:

- o vetor de estimativas de custo após a inicialização;
- o vetor de estimativas de custo após a 1ª iteração do algoritmo;
- o vetor de estimativas de custo após a 2ª iteração do algoritmo;
- os vértices explorados na 1ª e na 2ª iteração;
- o próximo vértice a ser explorado.

8. A matriz M a seguir representa um grafo.

Considere que as linhas e colunas da matriz seguem a ordem dos vértices:

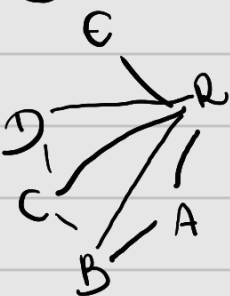
A, B, C, D, E, F, G, H .

$$M = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Com base nessa matriz, responda:

- Que tipo de grafo pode estar representado e qual representação está sendo utilizada?
- Desenhe o grafo correspondente à matriz.
- Conte o número de componentes conexas e o número de ciclos existentes no grafo.
- Desenhe uma representação do grafo usando listas de adjacências.

①



e-grau: 1

B-grau: 2

D-grau: 5

C-grau: 2

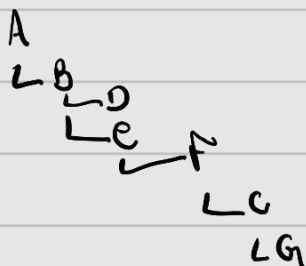
A-grau: 2

F-grau: 2

não é possível, pois todos os graus ímpares precisam ser par

②

15-9-2-1-4-7



③

② A, B, D, E, F, G, C

④

③ A-B-E-F-G

4

a) distância mínima H-G: 7

b) H-C-F-G

c)

H	A	B	C	D	E	F	G
0	∞	∞	∞	∞	∞	∞	∞
0	4	∞	4	∞	∞	0	∞
0	4	1	4	3	∞	∞	∞
0	4	1	4	3	3	1	∞
0	4	1	4	3	3	1	9
0	4	1	4	3	3	1	7

5) int contar_alcancaveis(Grafo *g, int origem) {

int visitados [g->maxvertices];

for (int i=0; i < g->n; i++) {

visitados[i] = 0;

}

int pilha [g->maxvertices];

int topo = 0;

int count = 0;

pilha [topo++] = origem;

visitados[origem] = 1;

while (topo > 0) {

int v = pilha [--topo];

count++;

for (int i=0; i < g->n; i++) {

if ($g \rightarrow m[v][i] == 1$ && !visited[i]) {

visited[i] = 1;

pilha[topo++] = i;

}

}

}

return count;

}

6

int eh-isolado (Grafo *g, int vertice) {

for (int i=0; i < g->n; i++) {

if (g->m[vertice][i] != 0) {

return 0;

}

}

return 1;

}

7

a A B C D E F G

∞ ∞ ∞ 0 ∞ ∞ ∞

d) vértices explorados D, F

e) próximo G

b A B C D E F G

∞ ∞ ∞ 0 ∞ ∞ ∞

5 9 ∞ 0 5 1 ∞

c A B C D E F G

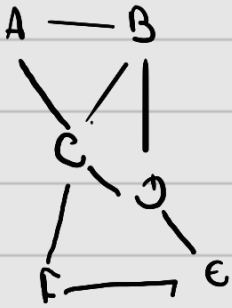
∞ ∞ ∞ 0 ∞ ∞ ∞

5 9 ∞ 0 5 1 ∞

5 9 ∞ 0 4 1 ∞

8) ²Tipos: grafos não direcionados (matriz simétrica) e não ponderados (0 e 1)
 Representação: matriz de adjacência

13)



G - H

© Componentes conexos: 2

ciclos: A - B - C - A

C - D - E - F - C

A - C - D - E - F - C - B - A

d)

vértice

adjacentes

A

B - C - NULL

B

A - C - D - NULL

C

A - B - D - F - NULL

D

B - C - E - NULL

E

D - F - NULL

F

C - E - NULL

G

H - NULL

H

G - NULL